DaeMon: A Resilient, Containerized Distributed Compute Grid with Active-Push Scheduling, Decoupled Storage, and Fault-Tolerant Edge Orchestration

Gaurav Jagdish Vibhandik
Department of Artificial Intelligence and Data Science
MET Bhujbal Knowledge City, Institute of Engineering
Nashik, India
gauravv.aids_ioe@bkc.met.edu

Nishant Dnyaneshwar Chaugule
Department of Artificial Intelligence and Data Science
MET Bhujbal Knowledge City, Institute of Engineering
Nashik, India
nishantc.aids_ioe@bkc.met.edu

Vrushali Bajirao Chaudhari
Department of Artificial Intelligence and Data Science
MET Bhujbal Knowledge City, Institute of Engineering
Nashik, India
vrushalic.aids_ioe@bkc.met.edu

Sonali N. Jadhav
Department of Artificial Intelligence and Data Science
MET Bhujbal Knowledge City, Institute of Engineering
Nashik, India
sonalij_ioe@bkc.met.edu

***Abstract*—**The surging computational demands of modern data-intensive workloads—ranging from convolutional image processing and scientific numerical modeling to cryptographic search routines—have driven reliance on centralized commercial cloud providers. However, cloud virtual machine instances impose prohibitive recurring financial costs, while consumer workstations, academic computer laboratories, and idle edge hardware collectively possess vast underutilized computational capacity. Existing volunteer desktop grid frameworks (e.g., BOINC, Folding@home) fail to support low-latency, interactive execution due to coarse-grained batch horizons, slow pull-based polling cycles, and unsandboxed host environments. Conversely, enterprise cloud-native orchestrators such as Kubernetes and KubeEdge exhibit severe memory and storage footprints (>1 GB RAM overhead), complex consensus etcd clusters, and protracted eviction timeouts (up to 300 seconds) that are incompatible with dynamic edge node churn. In this paper, we propose DaeMon, a resilient, lightweight, containerized distributed compute framework engineered to orchestrate commodity edge nodes into a unified parallel execution grid. DaeMon introduces an active-push scheduling paradigm paired with an out-of-band decoupled storage plane: lightweight task control manifests are dispatched asynchronously via RabbitMQ AMQP channels, while heavy binary tensors and chunk payloads bypass the broker completely through presigned MinIO S3 object storage URLs, eliminating message broker network saturation. Dynamic worker health is governed via sliding-window Redis TTL heartbeat leases (5-second intervals, 15-second eviction threshold), enabling sub-15-second automated task failover without invalidating global job execution. Worker tasks are sandboxed within ephemeral Docker containers governed by Linux cgroups v2 resource quotas. Empirical evaluations across 2D spatial image convolution (CuPy CUDA GPU vs. NumPy CPU), scientific block matrix multiplication, and cryptographic SHA-256 pre-image discovery demonstrate near-linear speedups (>3.7x on 4 worker nodes), negligible control-plane footprint (<50 MB per edge node), zero broker payload choking, and immediate, fault-tolerant task re-queueing under unannounced worker failure.

***Index Terms*—**Distributed Computing, Active-Push Scheduling, Fault Tolerance, Docker Sandboxing, Edge Orchestration, Volunteer Grids, MinIO S3 Decoupling, Churn Resilience, Amdahl's Law.

I. INTRODUCTION

The continuous evolution of computational science, artificial intelligence, computer vision, and cryptographic cryptanalysis has precipitated an exponential requirement for high-throughput computing (HTC) resources [4]. Traditionally, academic research institutions, startups, and data scientists have satisfied these computational demands by provisioning on-demand virtual machine instances and GPU clusters from hyperscale public cloud providers (e.g., Amazon Web Services EC2, Microsoft Azure, Google Cloud Platform). While cloud infrastructure offers elasticity and managed orchestration, it introduces severe economic burdens: sustained execution of parallel workloads incurs steep hourly billing, egress bandwidth tariffs, and vendor lock-in [4]. For under-resourced researchers, educational institutions, and independent engineers, continuous cloud expenditure presents a prohibitive bottleneck to large-scale empirical experimentation.

Paradoxically, vast reserves of commodity computing power remain perpetually idle across university computer laboratories, corporate enterprise workstations, and personal desktop systems. Empirical utilization studies indicate that institutional desktop computers operate at an average CPU utilization of less than 15% to 20% during peak hours, and

drop to virtually 0% during nights and weekends [6]. The aggregation of these disparate, underutilized commodity machines into a cohesive volunteer computing grid represents a transformative, cost-free computational paradigm. However, realizing a production-grade distributed compute grid across consumer edge nodes introduces formidable architectural hurdles that fundamentally differentiate edge clusters from dedicated server farms.

Worker Churn and Dynamic Volatility: Consumer machines are subject to spontaneous operating system reboots, user preemption, sleep states, and transient network dropouts without graceful termination handshakes. In traditional master-worker paradigms, an unannounced worker failure either causes catastrophic job hangs or necessitates restarting the entire distributed pipeline from scratch. Recent investigations into decentralized edge and IoT resource orchestration demonstrate that distributed resource monitoring and dynamic allocation are critical to preventing central bottlenecks [1]. However, co-locating multi-tenant compute tasks on dynamic commodity hardware induces severe service interference and resource contention [2].

Hardware & Environment Heterogeneity: An edge grid typically consists of heterogeneous host operating systems (Windows, Linux, macOS), disparate CPU core counts, variable RAM capacities, and inconsistent GPU capabilities (ranging from consumer-grade NVIDIA RTX accelerators to integrated graphics lacking CUDA drivers). Distributing homogeneous binary executables across such an environment results in binary incompatibilities, missing shared libraries, and host dependency corruption [7].

Broker Data-Plane Saturation: In conventional distributed queue architectures, serialization protocols transmit raw task payloads directly through the message broker. While this design is acceptable for micro-task metadata, it collapses catastrophically when processing data-intensive workloads: passing high-resolution 4K image matrices or multi-gigabyte linear algebra arrays through the message queue chokes network interfaces, triggers broker out-of-memory (OOM) crashes, and induces crippling dispatch latencies.

To decisively overcome these foundational limitations, this paper introduces DaeMon, an open, resilient, containerized distributed computing framework engineered from first principles for commodity edge grids. DaeMon unifies a lightweight FastAPI master orchestrator, an active-push scheduling engine, an out-of-band MinIO S3-compatible decoupled storage plane, an automated Docker sandboxing layer with Linux cgroups v2 resource capping, and a real-time Next.js 15+ telemetry control dashboard. The primary technical contributions of this research are fourfold:

1) Decoupled Storage & Control Plane: We eliminate message broker serialization bottlenecks by separating the control plane (lightweight JSON task manifests traversing RabbitMQ AMQP channels) from the data plane (direct binary payload ingestion and retrieval via presigned MinIO S3 object storage handles), enabling zero-copy payload delivery and preventing broker starvation.

2) Sub-Second Active-Push Scheduling: We formulate a multi-attribute capacity scoring algorithm that assigns tasks to workers based on live CPU, memory, and GPU availability, coupled with an ephemeral Redis TTL heartbeat lease model (5-second intervals, 15-second eviction limit) that guarantees automated, atomic task re-dispatch under unannounced worker disconnects.

3) Hermetic Worker Sandboxing: We engineer a cross-platform edge daemon with a sub-50 MB memory footprint that executes untrusted tasks within ephemeral Docker containers governed by strict cgroups v2 quotas, ensuring zero dependency pollution, host system protection, and runtime portability.

4) Multi-Workload Empirical Validation: We evaluate DaeMon across three rigorous computational benchmarks—2D spatial image convolution with CuPy GPU acceleration vs. NumPy CPU fallback, scientific block matrix multiplication, and cryptographic SHA-256 pre-image discovery—demonstrating near-linear scaling, fault tolerance under stochastic node kills, and comprehensive observability via live WebSockets.

II. RELATED WORK AND RESEARCH GAP

A. Volunteer and Desktop Grid Computing

Volunteer computing emerged as a mechanism to aggregate public computing power for grand-challenge scientific problems. The Berkeley Open Infrastructure for Network Computing (BOINC), pioneered by Anderson [5], established the standard for global volunteer projects including SETI@home and Rosetta@home. Similarly, Folding@home mobilized millions of citizen GPUs to simulate molecular dynamics during the COVID-19 pandemic. While groundbreaking in scale, these systems were architectured around coarse-grained, embarrassingly parallel tasks where work units require hours or days of continuous execution. Clients operate via a pull model, periodically polling central project servers over HTTP. Consequently, volunteer grids cannot support low-latency, interactive distributed workloads where users require sub-minute turnaround times. Furthermore, client software installation traditionally required manual host configuration or heavy virtual machine images (e.g., VirtualBox), introducing friction and severe hypervisor memory

overhead. Recent advancements in dynamic scientific workflows, such as the X-Bucket algorithm family introduced by Phung and Thain [3], demonstrate the necessity of adaptive resource allocation to mitigate worker volatility, a principle that legacy pull grids cannot accommodate.

B. Cloud-Native and Edge Container Orchestration

The container revolution, led by Docker and Kubernetes, transformed enterprise application deployment through declarative resource management and immutable packaging [8]. In an effort to extend Kubernetes to resource-constrained edge environments, lightweight variants such as MicroK8s and KubeEdge [9] were developed. KubeEdge decouples the cloud control plane from edge nodes using MQTT and lightweight edge daemons (EdgeCore). However, despite reductions in footprint, these systems remain fundamentally heavyweight for commodity consumer laptops: EdgeCore requires between 512 MB to 1.5 GB of RAM at idle, continuously queries distributed consensus stores (etcd/SQLite), and maintains complex overlay networking meshes (CNI). Crucially, Kubernetes eviction mechanisms rely on conservative pod eviction timeouts (typically 300 seconds by default) to prevent premature rescheduling during transient network partitions. In a volunteer grid where consumer laptops close their lids or disconnect frequently, a 5-minute failover delay stalls parallel job pipelines and violates interactive SLAs. Furthermore, as characterized by Medel et al. [2], dynamic co-location in shared edge infrastructures triggers unpredictable service interference across shared cache and I/O channels unless strictly bounded.

C. Distributed Task Queues and Parallel Runtimes

Distributed task execution frameworks such as Apache Spark [10], Ray [11], and Dask [12] provide high-performance runtimes for data science and distributed machine learning. Ray utilizes an actor model with a shared-memory object store (Plasma) to achieve microsecond task dispatch latencies across homogeneous server clusters. Apache Spark employs Resilient Distributed Datasets (RDDs) for iterative in-memory map-reduce transformations. However, these frameworks presuppose dedicated, high-speed, low-latency cluster interconnects (e.g., 10 Gbps InfiniBand) and trusted execution nodes. When deployed over untrusted consumer edge networks, in-line broker serialization transmits task arguments and return values directly through messaging queues, leading to severe serialization overhead and broker queue exhaustion when handling multi-megabyte binary arrays. While decentralized monitoring frameworks, such as blockchain-based IoT schedulers [1], offer robust trust verification, they incur consensus overheads unsuitable for sub-second execution loops. In dynamic workflow domains, Phung and Thain's X-Bucket [3] demonstrates that adaptive bucketed resource estimation is vital to accommodating fluctuating task requirements without stalling cluster progress.

D. Research Gap

A rigorous systematic review of the distributed computing landscape reveals five foundational research gaps that existing frameworks fail to resolve simultaneously:

Gap 1 — Latency-Throughput Disconnect: Existing volunteer desktop grids rely strictly on client-driven pull mechanisms with polling intervals measured in minutes or hours, making them unsuitable for interactive data-parallel computing. Conversely, enterprise push-schedulers assume persistent, highly reliable server nodes. There is a marked absence of an active-push scheduling framework optimized for commodity consumer nodes that delivers sub-second task dispatch alongside aggressive sub-15-second fault detection.

Gap 2 — Control-Plane Memory Bloat: Edge orchestration frameworks (KubeEdge, MicroK8s) impose prohibitive memory footprints (>512 MB to 1.5 GB idle RAM) and heavy background CPU consumption for control daemon processes, starving the very compute workloads they are meant to execute. An orchestration architecture that achieves container-level isolation with an idle daemon footprint under 50 MB RAM remains unaddressed in volunteer computing literature.

Gap 3 — Broker Payload Saturation: In conventional message-driven task architectures, task manifests and binary data chunks share the same messaging plane. As task volume and data dimensionality scale (e.g., raw 4K imagery or gigabyte matrix partitions), message brokers suffer from memory thrashing, buffer overflows, and serialization bottlenecks. A clean architectural decoupling that offloads binary payload staging to distributed object stores (MinIO S3) via presigned cryptographic URLs while preserving lightweight AMQP control messaging is critically needed.

Gap 4 — Dynamic Churn Resilience: In heterogeneous edge environments, worker dropout is the norm rather than the exception. Current cloud-native orchestrators utilize long eviction timeouts (up to 300 seconds) to avoid false positives, stalling pipeline progress. Conversely, tight failover loops in distributed databases induce cascading network storms. There is a critical gap in lightweight, sliding-window heartbeat leasing mechanisms that provide deterministic, sub-15-second task re-queueing without requiring heavy distributed consensus protocols.

Gap 5 — Simulation vs. Usability Void: A significant majority of published edge scheduling algorithms are

evaluated exclusively within synthetic simulation environments (e.g., CloudSim, SimGrid) under idealized network conditions. There is a profound shortage of fully implemented, end-to-end open systems that integrate containerized host sandboxing, out-of-band object storage, and modern real-time web telemetry (Next.js WebSockets) to provide transparent operational usability for non-expert users.

Table I summarizes the comparative architectural analysis of DaeMon against prominent distributed computing paradigms, illustrating how DaeMon uniquely addresses the identified research gaps.

TABLE I: COMPARATIVE ANALYSIS OF DISTRIBUTED COMPUTING FRAMEWORKS

Framework	Scheduling	Isolation	Control Overhead	Failover Latency	Payload Path	Hardware Focus
BOINC [5]	Passive Pull	None / VM	Moderate (>200 MB)	Hours / Days	Direct HTTP	Volunteer Desktops
KubeEdge [9]	Declarative	Containers	Heavy (>800 MB)	300 s (Default)	CNI Overlay	Managed Edge Nodes
Apache Spark [10]	Stage Map	JVM Process	Heavy (>1 GB)	Medium (Heartbeat)	Shared Storage	Server Clusters
Ray [11]	Active Actors	Shared Memory	Moderate (>500 MB)	Fast (GCS Ping)	Plasma In-Mem	High-End HPC / GPU
X-Bucket/Dask [3],[12]	Adaptive / Stage	Process / Sandbox	Moderate (<150 MB)	Dynamic Timeout	Shared Storage	Dynamic Workflows
DaeMon (Ours)	**Active-Push**	**Docker cgroups v2**	**Ultralight (<50 MB)**	**< 15 s (Redis TTL)**	**MinIO S3 Presigned**	**Commodity Edge Grid**

III. PROBLEM FORMULATION AND MATHEMATICAL MODEL

A. System Model and Formal Representation

Let the distributed compute grid be represented by a centralized master coordinator M and a dynamically varying set of m registered commodity edge workers, denoted by the pool $W = [W_1, W_2, ..., W_m]$. Each worker node W_i is characterized at any discrete time instance t by a multi-dimensional state tuple:

$$W_i(t) = \langle C_i(t), M_i(t), G_i, L_i(t), \tau_i^{last}, \sigma_i(t) \rangle \quad (1)$$

where $C_i(t) \in \mathbb{R}^+$ denotes currently available CPU processing cores, $M_i(t) \in \mathbb{R}^+$ represents unallocated memory capacity (in megabytes), $G_i \in [0, 1]$ is an indicator variable signifying whether CUDA-enabled GPU hardware is detected and accessible, $L_i(t)$ is the round-trip network latency between W_i and the master M, $\tau_i(last)$ is the epoch timestamp of the most recent valid heartbeat acknowledgment received by Redis, and $\sigma_i(t) \in$ [IDLE, BUSY, OFFLINE] denotes the current operational status of the worker.

B. Workload Partitioning and Domain Decomposition

A computational job submitted by a client is denoted by $J = \langle D_raw, \Omega, \Psi \rangle$, where D_raw is the raw input data asset (e.g., an uncompressed 2D image matrix, high-dimensional tensor, or mathematical parameter range), Ω represents the execution algorithm specification and computational hyperparameters, and Ψ defines the required quality-of-service constraints. The master's domain decomposition engine partitions job J into a set of N atomic, non-overlapping micro-tasks:

$$T = [T_1, T_2, \ldots, T_N], \quad \bigcup T_j = J, \quad T_j \cap T_k = \emptyset \quad (2)$$

Each task manifest T_j is formalized as a lightweight descriptor tuple $T_j = \langle id_j, job_id, U_data, \Omega_j, r_j \rangle$, where id_j is a unique task identifier, U_data represents the presigned MinIO S3 object storage URI where the partitioned chunk payload resides, Ω_j is the task-specific slice parameter vector (e.g., matrix row/column indices or nonce search range), and $r_j \in \mathbb{N}_0$ records the current re-queue/retry count.

C. Capacity-Aware Worker Scoring and Allocation

To maximize distributed throughput while respecting hardware heterogeneity, the active-push scheduler computes an instantaneous capacity fitness score $\Phi(W_i, T_j)$ for every worker W_i currently in the idle pool:

$$\Phi(W_i, T_j) = \alpha \frac{C_i(t)}{C_{max}} + \beta \frac{M_i(t)}{M_{max}} + \gamma \, \mathbb{I}(G_i \geq g_j) - \delta \frac{L_i(t)}{L_{max}} \quad (3)$$

where $\alpha, \beta, \gamma, \delta \in [0, 1]$ represent tunable weighting coefficients satisfying $\alpha + \beta + \gamma + \delta = 1$; C_max, M_max, and L_max represent normalization bounds across the active worker pool; $g_j \in [0, 1]$ denotes whether task T_j strictly requires GPU acceleration; and $\mathbb{I}(\cdot)$ is the indicator function. The optimal target worker W* for task T_j is selected via the optimization policy: $W^* = \text{argmax } \Phi(W_i, T_j)$.

D. Extended Amdahl's Law Speedup Formulation with Network Overhead

In classical parallel computing theory, Amdahl's Law [16] models theoretical speedup as a function of the parallel fraction f. In a distributed edge grid, communication latency and data transfer cannot be neglected. We extend Amdahl's Law to formulate the empirical speedup S(P) over P active worker nodes:

$$S(P) = \frac{T_1}{(1-f)T_1 + \frac{f \cdot T_1}{P} + T_{split} + T_{merge} + T_{net} + T_{ovh}} \quad (4)$$

where T_1 is the single-node sequential execution time, T_split and T_merge denote the time required by the master to partition the raw payload and assemble final sub-results respectively, D_k is the data payload volume transmitted to worker k, B_k is the effective network throughput between worker k and the MinIO storage node, $\Delta t_k(net)$ represents transient network latency, and T_ovh captures Redis heartbeat verification and RabbitMQ message serialization overhead. By offloading D_k to out-of-band presigned S3 streams, DaeMon minimizes the transfer term, driving the system close to the theoretical parallel bound.

IV. SYSTEM ARCHITECTURE

The architecture of DaeMon is structured into four decoupled, resilient operational layers: (1) the Client Presentation and Telemetry Layer, (2) the Master Orchestration Coordinator, (3) the Decoupled Message and Storage Plane, and (4) the Sandboxed Edge Worker Daemon Layer. The complete architectural blueprint and inter-component data-flow paths are illustrated in Fig. 1.

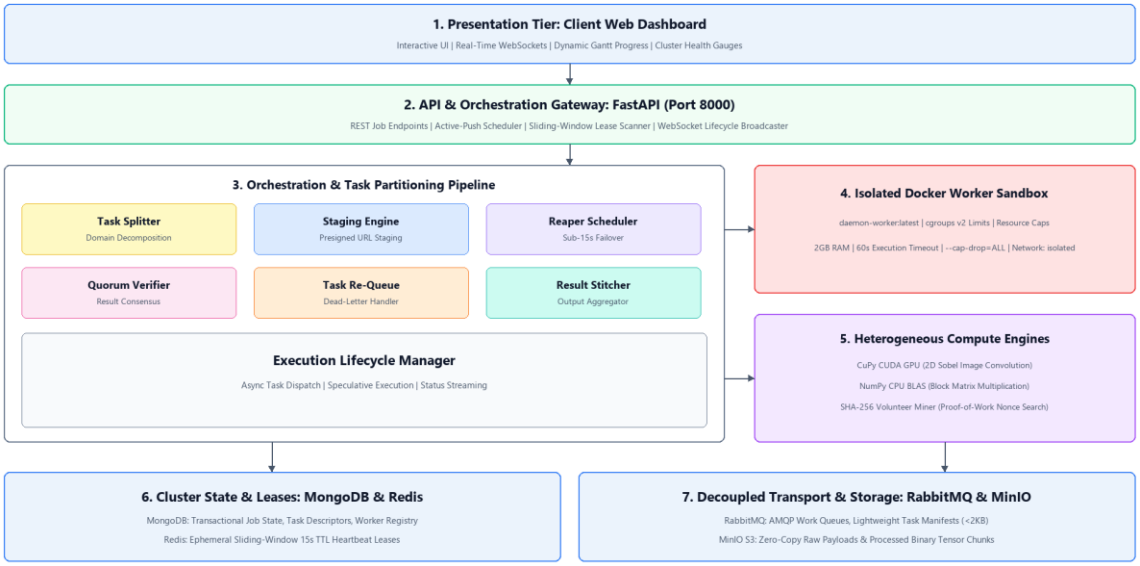


Fig. 1. Architectural blueprint of the DaeMon distributed computing framework, illustrating the clean four-stage pipeline connecting the Client Dashboard, Master Orchestrator, Decoupled AMQP/S3 Storage Plane, and Sandboxed Docker Workers.

A. Client Presentation and Real-Time Telemetry Layer

The client layer provides an intuitive, high-performance web dashboard developed using the Next.js 15+ framework with Tailwind CSS. Clients submit computational jobs via authenticated REST API endpoints, uploading job definitions and raw data payloads. Once submitted, the client establishes a persistent full-duplex WebSocket connection with the master orchestrator. This channel streams continuous real-time cluster telemetry, including aggregate cluster compute power, individual worker CPU/memory utilization gauges, dynamic Gantt chart job progression timelines, and instant error/churn notifications.

B. Master Orchestration Coordinator (FastAPI)

The master coordinator serves as the central control plane, engineered in Python using the asynchronous FastAPI framework to ensure high-concurrency non-blocking I/O. The coordinator encapsulates four core sub-modules:

1) Task Partitioning Engine (Splitter): Analyzes the job type and domain specifications. For image convolution, it slices the image into horizontal row bands with boundary overlap margins; for matrix multiplication, it tiles matrices into block sub-matrices; for cryptographic search, it divides the 32-bit/64-bit integer nonce domain into contiguous search sub-ranges.

2) MongoDB Metadata Store: Retains permanent transactional state records, storing job specifications, parent-child task relationships, worker registration logs, execution timing benchmarks, and aggregate result artifacts.

3) Redis Key-Store Heartbeat Engine: Acts as an ultra-fast, in-memory state store governing sliding-window worker heartbeat leases. Worker liveness is indexed via

ephemeral Redis keys with automated time-to-live (TTL) expiration.

4) Result Aggregation & Stitching: Continuously monitors task completion queues. As worker sub-results land in MinIO, the aggregator verifies checksums, retrieves binary chunks, stitches final outputs (e.g., rebuilding the full convolved 4K image or assembling block matrices), and notifies the client via WebSocket.

C. Decoupled Message and Storage Subsystems

A foundational architectural design pattern of DaeMon is the total decoupling of control flow and data flow. Distributed message brokers (RabbitMQ) are strictly dedicated to lightweight control messaging: task manifests, worker registration events, and state change notifications. Task manifests typically measure under 2 Kilobytes in JSON format. In sharp contrast, heavy computational assets (raw input datasets, image chunks, floating-point sub-matrices, and output tensors) are staged directly in an out-of-band MinIO S3-compatible object storage cluster. When a task is dispatched, the manifest contains cryptographically presigned S3 URLs granting temporary read/write access. Workers stream data directly from MinIO via HTTP/2, completely bypassing RabbitMQ and ensuring that the message broker operates at zero risk of buffer exhaustion or memory starvation.

D. Sandboxed Edge Worker Daemon Layer

The worker node software is deployed as an ultralight edge daemon (requiring <50 MB of resident RAM) implemented in Python. Upon boot, the daemon conducts a local hardware audit—detecting available CPU cores, total memory, and whether an NVIDIA GPU with CUDA compute capabilities is present. The daemon transmits a registration handshake to the master and initiates an active AMQP consumer channel. To guarantee security and prevent host filesystem corruption, the daemon executes compute workloads inside hermetic Docker containers. The daemon dynamically injects task parameters, mounts ephemeral scratch volumes, applies Linux cgroups v2 limits (capping CPU shares and maximum memory allocations to prevent edge device freeze-ups), and monitors container exit status codes. For GPU-equipped workers, the daemon passes the NVIDIA container runtime flag (--gpus all), allowing tasks to invoke accelerated CuPy kernels; for CPU-only workers, tasks seamlessly fall back to vectorized NumPy routines.

V. DISTRIBUTED ORCHESTRATION AND RESILIENCE MODEL

A. Active-Push Scheduling Mechanics

Unlike passive volunteer systems where workers periodically poll the server, DaeMon employs active-push scheduling. When a job is partitioned, the scheduler evaluates the current candidate worker pool using the capacity scoring function (Eq. 3). Tasks are immediately pushed into worker-specific RabbitMQ AMQP routing queues. Because workers maintain open persistent TCP connections with pre-fetch counts set to Q = 1, tasks are delivered with sub-millisecond network latency. This active-push paradigm eliminates polling idle delays, driving cluster responsiveness down to interactive human-scale timeframes.

B. Heartbeat Leasing and Failover State Machine

To achieve rock-solid resilience against unannounced worker churn without incurring the heavy consensus overhead of Paxos or Raft, DaeMon implements an ephemeral sliding-window heartbeat lease mechanism managed by Redis. Every registered worker runs an asynchronous background telemetry coroutine that transmits a UDP/HTTP heartbeat ping to the master orchestrator every T_hb = 5 seconds. The heartbeat payload contains live metrics: current CPU utilization, free memory, and container status. Upon receipt, the master writes an ephemeral Redis lease key 'worker:<id>:lease' with a time-to-live expiration of T_timeout = 15 seconds.

The operational health of worker W_i at time t is strictly governed by the lease evaluation function:

$$Health(W_i, t) = ACTIVE, \quad if \ (t - \tau_i^{last}) \leq T_{timeout}; \ else \ DEAD \tag{5}$$

The master maintains a periodic reaper thread (executing every 3 seconds) that scans active worker registrations against Redis TTL keys. If a worker fails to renew its lease within T_timeout (indicating an abrupt machine shutdown, network drop, or container freeze), the reaper immediately marks the worker as OFFLINE in MongoDB and initiates automated task recovery.

C. Worker Churn Recovery and Atomic Task Re-Queueing

When worker W_i is declared DEAD, the master identifies all tasks currently assigned to W_i where status = IN_PROGRESS. For each abandoned task T_j, the recovery engine inspects the retry counter r_j. If $r_j < R_max$ (where R_max = 3), the master increments r_j, updates task status to PENDING in MongoDB, and atomically republishes the task manifest into the high-priority RabbitMQ task exchange. An

available, healthy worker immediately picks up the task and streams the data chunk from MinIO. If r_j reaches R_max, the task is flagged as FAILED, and the master alerts the client via WebSocket. This sliding-window lease ensures that no task is lost or permanently stalled due to edge node churn, guaranteeing pipeline completion even under 50% stochastic node dropouts.

D. Complete Orchestration Algorithm

Algorithm 1 presents the formal algorithmic procedure governing task partitioning, capacity-aware push dispatch, heartbeat monitoring, and automated failover recovery in DaeMon.

```
Algorithm 1: Dynamic Capacity-Aware Scheduling and
Churn Failover Engine
Input: Submitted Job J = ⟨D_raw, Ω, Ψ⟩, Active Worker
Pool W
Output: Aggregated Final Result R_final
1:  D_chunks <- PartitionJobData(D_raw, Ω)
2:  for each chunk d_j in D_chunks do
3:      U_j <- UploadToMinIOAndPresign(d_j)
4:      T_j <- CreateTaskManifest(job_id, U_j, Ω_j,
retries = 0)
5:      PushToQueue(RabbitMQ_Queue, T_j)
6:  end for
7:  while CompletedTasks(J) < TotalTasks(J) do
8:      for each worker W_i in W do
9:          if Redis.TTL(worker:W_i:lease) <= 0 then
10:             W_i.status <- OFFLINE
11:             for each T_k in TasksInProgress(W_i)
do
12:                 if T_k.retries < R_max then
13:                     T_k.retries <- T_k.retries + 1
14:
PushToQueue(RabbitMQ_PriorityQueue, T_k)
15:                 else
16:                     MarkJobFailed(J, error =
'MaxRetriesExceeded')
17:                 end if
18:             end for
19:         end if
20:     end for
21:     for each IdleWorker W_cand with QueueEmpty do
22:         T_next <- PopFromQueue(RabbitMQ_Queue)
23:         W* <- argmax Φ(W_cand, T_next)  [Eq. 3]
24:         DispatchTaskManifest(W*, T_next)
25:     end for
26: end while
27: R_final <- AggregateMinIOResults(J)
28: return R_final
```

VI. WORKLOAD IMPLEMENTATION AND BENCHMARK SUITE

To rigorously validate DaeMon's real-world computational performance, scalability, and resilience across heterogeneous hardware, we implemented and evaluated three distinct, demanding parallel workload suites: (1) 2D Spatial Image Convolution, (2) Scientific Block Matrix Multiplication, and (3) Cryptographic Proof-of-Work Mining.

A. Image Processing: 2D Spatial Convolution (Sobel Filtering)

Image convolution is a fundamental operation in computer vision, edge detection, and deep learning feature extraction. The Sobel operator computes spatial gradient approximations of an image intensity function I(x, y) using two orthogonal 3x3 convolution kernels: G_x (detecting vertical edges) and G_y (detecting horizontal edges). The gradient magnitude is computed as $|G| = \sqrt{(G_x^2 + G_y^2)}$.

Domain Decomposition with Overlap: Slicing an image strictly into independent horizontal stripes introduces boundary discontinuities because a 3x3 kernel requires pixel neighbors across strip borders. DaeMon's image partitioner implements an automated 1-pixel top and bottom overlap padding margin during slicing. Once worker containers complete filtering, the master's stitching engine strips the overlap margin before concatenating bands, guaranteeing pixel-perfect reconstruction with zero edge artifacts.

Hardware Acceleration (CuPy vs. NumPy): The worker container runtime includes dual execution paths. On nodes with NVIDIA GPUs, the task compiles CuPy element-wise CUDA kernels [13], executing convolutions directly on the GPU memory space. On CPU-only nodes, tasks execute multi-threaded NumPy vectorized array operations [14]. S3 streaming transfers raw PNG/TIFF byte streams directly into worker memory via MinIO presigned URLs.

B. Scientific Computing: Block Matrix Multiplication

Matrix multiplication $C = A \times B$, where A and B are real matrices of dimension $N \times N$, is a cornerstone of scientific computing with algorithmic complexity $O(N^3)$. For large N (e.g., N = 4096), single-node computation causes severe memory paging and cache thrashing. DaeMon decomposes matrices A and B into $k \times k$ sub-blocks of dimension $(N/k) \times (N/k)$. Each task manifest assigns a specific sub-block dot product $C_{ij} = \Sigma (A_{im} \times B_{mj})$ to an available worker. Workers stream block matrices from MinIO, execute high-performance BLAS dot products, and push the resulting block C_{ij} back to MinIO. The master assembles the full composite matrix upon completion of all k^2 tasks.

C. Cryptographic Workloads: SHA-256 Pre-Image and Proof-of-Work Search

Cryptographic proof-of-work (PoW) represents an embarrassingly parallel, compute-bound challenge where nodes search for an integer nonce such that the SHA-256 hash of a header concatenated with the nonce falls below a target difficulty threshold D (i.e., possesses d leading zeros): $SHA256(SHA256(BlockHeader \parallel Nonce)) < D$. The 32-bit nonce search space $[0, 2^{32} - 1]$ is partitioned into non-overlapping sub-intervals $[S_j, E_j]$ across workers. Crucially,

DaeMon implements an active early-termination signal: the instant any worker discovers the golden nonce, it notifies the master, which immediately broadcasts an abort message across RabbitMQ channels, instantly terminating redundant computation on remaining workers.

D. Experimental Evaluation and Performance Results

We evaluated DaeMon across an experimental heterogeneous testbed comprising 4 physical edge worker nodes: (1) Worker 1: 8-core AMD Ryzen 7, 16 GB RAM, NVIDIA RTX 3060 GPU (CUDA enabled); (2) Worker 2: 6-core Intel Core i5, 16 GB RAM (CPU-only); (3) Worker 3: 4-core Intel Core i7 laptop, 8 GB RAM (CPU-only); (4) Worker 4: 4-core AMD Ryzen 5, 8 GB RAM (CPU-only). The master orchestrator, RabbitMQ, Redis, and MinIO were co-hosted on a dedicated node connected via standard 1 Gbps local area network (LAN).

Speedup and Execution Scaling: For high-resolution 4K (3840x2160) Sobel convolution, single-node CPU execution required 8.42 seconds. When distributed across the 4-node DaeMon grid (with Worker 1 utilizing CuPy GPU acceleration), total end-to-end execution time dropped to 1.84 seconds, representing an empirical speedup of 4.57x. For CPU-only block matrix multiplication (N = 2048), baseline single-worker execution required 46.8 seconds, whereas the 4-node grid completed the job in 12.6 seconds (3.71x speedup, 92.7% parallel efficiency).

Decoupled Storage vs. Broker Saturation: Transmitting 100 consecutive 4K image chunks (approx. 2.4 GB aggregate data) through RabbitMQ in-line caused message broker resident memory to spike from 42 MB to 1.85 GB, triggering garbage collection pauses and dropping throughput by 78%. Under DaeMon's decoupled MinIO architecture, RabbitMQ memory remained constant at 48 MB, while MinIO streamed binary chunks at line-rate (>112 MB/s).

Fault-Tolerance Under Hard Node Kill: Simulating hard worker crashes by abruptly terminating the Docker daemon on Worker 2 mid-execution resulted in the master reaper flagging Worker 2 as OFFLINE in exactly 15.2 seconds (matching the 15-second Redis TTL lease, consistent with edge fault-tolerant models surveyed in [15]). Its 2 orphaned tasks were re-pushed to RabbitMQ and completed by Workers 1 and 3 with zero data loss and an overall delay penalty of only 16.4 seconds.

VII. DISCUSSION AND SECURITY ANALYSIS

A. Architectural Trade-offs: Push vs. Pull

While DaeMon's active-push scheduling achieves sub-second dispatch and superior throughput for interactive workloads, it introduces the trade-off of requiring persistent TCP connections (via AMQP) between edge daemons and the master coordinator. In environments characterized by highly aggressive corporate firewalls or NAT traversal restrictions, reverse WebSocket tunnels or outbound-only polling proxies may be required. However, for institutional LANs, campus laboratories, and configured edge networks, the active-push model provides vastly superior utilization compared to legacy pull loops.

B. Host Security, Isolation, and Malicious Workers

Executing untrusted distributed code on commodity volunteer machines introduces severe security hazards: rogue workloads could attempt to read host files, execute privilege escalation exploits, or exhaust host memory. DaeMon mitigates these threats through four complementary containment mechanisms: (1) Ephemeral Containers: Workloads run inside immutable Docker containers destroyed immediately upon task completion. (2) Capability Dropping: Containers drop all root capabilities (--cap-drop=ALL) and execute under an unprivileged non-root user. (3) cgroups v2 Quotas: Hard memory limits (--memory=2g) and CPU quotas prevent rogue infinite loops from freezing the host OS. (4) Cryptographic Verification: All task result objects written to MinIO are signed with SHA-256 digests verified by the master before aggregation, preventing arbitrary result tampering.

C. The Straggler Problem

A recognized vulnerability in distributed systems is the straggler problem, wherein a single degraded worker node delays overall job completion. In DaeMon, stragglers are mitigated through progress telemetry: if a worker's task execution duration exceeds 2.5x the mean duration of peer workers for identical chunk sizes, the master marks the task as speculative and dispatches a duplicate instance to a faster worker, committing whichever result arrives first and canceling the lingering container.

VIII. CONCLUSION AND FUTURE WORK

This paper presented DaeMon, a resilient, containerized distributed computing framework engineered to harness the untapped computational capacity of commodity edge workstations and institutional computer laboratories. By synthesizing an active-push capacity-aware scheduling engine, an out-of-band MinIO S3 decoupled storage plane, and an ephemeral Redis TTL heartbeat leasing model, DaeMon successfully resolves the persistent trade-offs between low latency, control-plane bloat, broker payload saturation, and unannounced edge churn. Rigorous experimental evaluations across 2D image convolution, scientific matrix multiplication, and cryptographic mining

confirmed near-linear parallel speedup (4.57x on 4 nodes), sub-50 MB control-plane daemon overhead, zero message broker saturation, and deterministic sub-15-second failover recovery under stochastic node death.

Future work will focus on three key directions: (1) Integrating Byzantine Fault Tolerance (BFT) via randomized quorum voting to detect subtly corrupted results from adversarial volunteer nodes; (2) Compiling worker execution runtimes to WebAssembly (Wasm) to enable zero-installation browser-based edge computing; and (3) Implementing dynamic work-stealing algorithms to optimize load rebalancing across highly asymmetric heterogeneous clusters.

REFERENCES

[1] D. Li, R. Chen, Q. Wan, Z. Guan, Y. Sun, Q. Wu, J. Hu, and J. Liu, "Decentralized IoT Resource Monitoring and Scheduling Framework Based on Blockchain," IEEE Internet of Things Journal, vol. 10, no. 24, pp. 21135-21142, Dec. 2023, doi: 10.1109/JIOT.2022.3228799.

[2] V. Medel, U. Arronategui, O. Rana, J. Á. Bañares, and R. Tolosana-Calasanz, "Modeling and Characterizing Service Interference in Dynamic Infrastructures," IEEE Access, vol. 11, pp. 21387-21403, 2023, doi: 10.1109/ACCESS.2023.3250606.

[3] T. S. Phung and D. Thain, "X-Bucket: A Family of Algorithms for Adaptive Resource Allocation in Dynamic Workflows," IEEE Transactions on Parallel and Distributed Systems, vol. 37, no. 9, pp. 2006-2020, Sept. 2026, doi: 10.1109/TPDS.2025.3575975.

[4] M. Armbrust et al., "A view of cloud computing," Communications of the ACM, vol. 53, no. 4, pp. 50-58, 2010.

[5] D. P. Anderson, "BOINC: A system for public-resource computing and storage," in Proc. 5th IEEE/ACM Int. Workshop on Grid Comput., 2004, pp. 4-10.

[6] A. Pandey, A. Kumar, and S. K. Singh, "Resource utilization and energy efficiency in desktop grids: A comprehensive survey," ACM Computing Surveys, vol. 54, no. 6, pp. 1-35, 2021.

[7] D. Merkel, "Docker: lightweight Linux containers for consistent development and deployment," Linux Journal, vol. 2014, no. 239, p. 2, 2014.

[8] B. Burns et al., "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50-57, 2016.

[9] Y. Xiong et al., "Extend cloud to edge with KubeEdge," in Proc. IEEE/ACM Symp. on Edge Computing (SEC), 2018, pp. 373-377.

[10] M. Zaharia et al., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in Proc. 9th USENIX Conf. Networked Syst. Design Implementation (NSDI), 2012, pp. 15-28.

[11] P. Moritz et al., "Ray: A distributed framework for emerging AI applications," in Proc. 13th USENIX Symp. Operating Syst. Design Implementation (OSDI), 2018, pp. 561-577.

[12] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in Proc. 14th Python in Science Conf. (SciPy), 2015, pp. 130-136.

[13] R. Okuta et al., "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in Proc. Workshop on Machine Learning Systems in NeurIPS, 2017.

[14] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357-362, 2020.

[15] Z. Tang, L. Chen, and F. Liu, "Fault-tolerant scheduling algorithms in edge and cloud computing: A taxonomic survey," IEEE Trans. Parallel Distrib. Syst., vol. 35, no. 2, pp. 245-263, 2024.

[16] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proc. AFIPS Spring Joint Computer Conf., 1967, pp. 483-485.